

What Is Actually Verifiable in a Published Web Artifact

Two Contract Classes and One Feasibility Study

ANONYMOUS AUTHOR(S)

Which properties of a published web artifact can executable contracts establish, and where do their guarantees stop? We study two distinct verification boundaries and one exploratory generation pipeline, not a controlled comparison of cost-effectiveness. For **geometry**, finite browser probes on 1,200 pages find no sibling coupling in 45.76% of 2,255 eligible flex containers. This observed oracle is an upper bound for the same perturbation contract, not a proof of independence. An item-local predicate admits every item in 28.07% of containers, with 100.00% observed precision over 3,004 admissions and 78.15% recall. Exact search establishes soundness and maximality on eight checked finite domains totaling 3.5M configuration–item pairs; a model-checked witness certificate supports the baseline domain. We do not prove maximality for arbitrary item counts or values. For **disclosure**, one repository demonstrates two coverage defects: reversed contact information missed by a literal assertion, and two published PDFs outside the existing audit set. A shared policy couples abstract checking with encoding-aware inspection of emitted text, including rechecking declared removals. For **generated artifacts**, one paid feasibility run consumed 13 model requests and 349,968 recorded tokens over 49 minutes 45 seconds. Its fresh artifact received a local baseline rating of 58/100, while the formal arm remained blocked before application repair and received no score. In a synthetic offline control, TLC and ordinary deeper graph search generated the same 12-action defect-revealing trace. This establishes neither an artifact-quality gain nor an incremental defect-detection advantage from TLA+. The contribution is a measured account of coverage, failure, and assumptions rather than a favorable ranking.

Additional Key Words and Phrases: compositional verification, information flow control, TLA+, CSS layout, published artifacts

1 Introduction

A public web artifact is read by people, crawlers, assistive technology, language models, and archives. Treating it as governed by executable contracts raises an engineering question: *which properties can be discharged mechanically, over which inputs, and with what specification obligations?*

The answer matters because a passing check can conceal either a restricted property or incomplete inputs. Layout independence is stronger than the absence of an observed layout defect. Likewise, an absence of known strings in a declared sink is weaker than information-flow security. Comparing these checks requires making those differences explicit, and execution time alone does not measure the human work of specifying and maintaining them.

We study two contract classes in web publication, on different empirical scales: a rendered-page corpus for geometry and one repository for disclosure. A third exploratory study asks whether generated behavioral specifications and TLA+ feedback improve one LLM-generated web artifact. It records a failed feasibility run, not an efficacy estimate. None of the studies establishes a comparative return on engineering effort.

Geometric contracts (§3) ask when an item’s used main size is independent of sibling changes, a sufficient route to decomposing the size calculation. Modular layout proofs using rely/guarantee assumptions cover a different fragment [1]; independence is not a necessary condition for every such proof. We measure an observed upper bound for our particular contract, then compare an item-local predicate against it. That predicate is maximal on the checked domains, not proved optimal over arbitrary CSS or richer contract vocabularies.

Disclosure contracts (§4) use lattice labels [5] to describe permitted publication, with a narrower concrete checker for declared PII flows. The design contribution is to preserve labels across reversible encodings and recheck a declared removal against emitted text. A policy declaration cannot by itself

turn a detectable value into a successful removal. This does not constitute a new information-flow calculus or a proof that all sensitive information was destroyed.

Our contributions are: (i) an item-local independence predicate with soundness and maximality established on eight finite domains, plus checked rejection witnesses on the baseline; (ii) a separation of observed container independence (45.76%) from predicate coverage (28.07%), with a descriptive breakdown of the 840 missed items; (iii) a disclosure-checking design and single-repository case, including its policy inventory; (iv) a negative feasibility result for generated formal feedback, including its cost and blocked evidence; and (v) defects in our own instruments and models. These results support scrutiny of verification boundaries, not a recommendation that formal feedback improves generated artifacts.

2 Method and Instruments

Both studies use the same discipline, which we state because it produced most of the corrections in §5.

Fidelity and falsification. Both numerical checks concern Study A: recovered inputs must reproduce browser sizes, and an admitted item observed to move is a counterexample to applying the predicate in that setting. The saved score uses a 0.5 px fidelity tolerance and retains eight mismatches rather than satisfying a literal zero-mismatch gate. We report those exceptions instead of describing the instrument as exact on the corpus. Study B separately tests whether policy mutants can falsify its invariants and whether emitted text contradicts declared removals; neither establishes complete source or flow recovery.

Three instruments, by what each is good at. TLC [9] finds counterexamples on small domains. An exact-rational implementation enumerates finite configuration spaces without the rounding introduced by integer division in the TLA+ model. Chromium, driven through Playwright [10], supplies observations under a finite probing protocol. Agreement supports the model on those cases; it is not an unbounded proof. Disagreements require attribution rather than averaging away.

Corpus and perturbations. We render 1,200 pages from a public 2.56M-page web corpus [4] at widths 320, 480, 768, 1024, 1280, and 1920 pixels. The protocol pins container dimensions and perturbs each sibling by appending a 300 px probe and, separately, clearing its contents; movement above 0.5 px marks coupling. These content edits are not exhaustive changes to flex bases. External requests are blocked, which controls network variability but changes missing-image layout and does not ensure cross-machine identity. Inputs and labels come from the same altered rendering. The score excludes 238 zero-size containers from 2,493 recorded containers, leaving 2,255; all reported container percentages use that denominator.

Transition-safe measurement (D3). An active CSS transition has cascade priority over inline !important. Writing a probe value can therefore start a transition and an immediate read can return an old or intermediate value instead of the intended one. A sentinel-colour write/read exposed this failure in our measurement process. Disabling transitions and animations and verifying that each probe restores the baseline are measurement controls, not cosmetic choices. The saved results do not contain a paired transition-on/off run or an affected-page count, so we cannot attribute a numerical error magnitude to D3 alone. We promote the failure mechanism here because it applies to browser instrumentation beyond this study.

3 Study A: Geometric Contracts

3.1 The question

We investigate one route to compositional reasoning: independence of an item’s used main size under sibling perturbations, with container dimensions fixed. In the finite model, the perturbations replace

any one sibling's flex base by another value in the configured domain, at each configured width. In the browser, the finite content probes in §2 supply an empirical approximation. Neither contract covers every change to authored CSS. Our *item-local* vocabulary contains only the item's basis, grow and shrink factors, and used bounds; it excludes container aggregates. A richer rely/guarantee contract may instead describe dependencies between components, so failure of this independence test does not rule out compositional verification in general.

3.2 The condition

Let $\text{Hyp}(i)$ be the hypothetical main size, the flex base size clamped by the item's own bounds, as used by the flexible-length resolution algorithm [3]. An absent maximum is $+\infty$. Then

$$\begin{aligned}\text{CannotGrow}(i) &\equiv g_i = 0 \vee \text{Hyp}(i) \geq \max_i \\ \text{CannotShrink}(i) &\equiv s_i = 0 \vee \text{Hyp}(i) \leq \min_i \\ \text{Immobile}(i) &\equiv \text{CannotGrow}(i) \wedge \text{CannotShrink}(i)\end{aligned}$$

The predicate reads no sibling, container width, or grow/shrink mode. Mode depends on the sum of every item's hypothetical size, so a sibling can flip it; TLC refuted a mode-conditioned candidate. Omitting width makes the predicate width-independent only for a fixed tuple of used inputs. Responsive CSS can change that tuple, so the browser scorer requires admission at all six measured widths. Absence of a width variable does not itself prove soundness of the predicate or independence over an unsampled interval.

3.3 Maximality on the checked domains

Fix a finite domain D , its configurations, widths, and allowed sibling edits. Define A_D to contain exactly those item shapes that are independent in every configuration in D carrying that shape. Any sound predicate restricted to observing the shape must admit a subset of A_D ; admitting A_D itself is sound on D . This characterizes the optimal local classifier by definition. The substantive, separately checked assertion is that Immobile coincides with A_D . We establish that equality by enumeration for the eight domains below, not by induction for arbitrary N or arbitrary numerical values.

On the baseline domain (48 shapes, 110,592 configurations, 331,776 configuration–item pairs) there are 20 always-independent shapes, 12 sometimes-independent, and 16 never-independent. Immobile admits exactly the 20 always-independent shapes. The 12 sometimes-independent shapes cannot be admitted by a sound shape-only predicate on this domain; they account for 2.30% of its independent items. That finite-domain residue is not a bound on the browser corpus.

For each of the 28 rejected baseline shapes, the certificate supplies a concrete configuration and sibling edit that moves the item. TLC checks these rejection witnesses (recorded runtime 1.2 seconds); together with soundness of the admitted set on the baseline, they certify maximality there. Integer-model witnesses are cross-checked against the exact-rational implementation. The full baseline soundness check takes 8 minutes 21 seconds. Neither the certificate nor that runtime represents a proof for unbounded CSS inputs.

3.4 Bound sensitivity

Bounded checking establishes a bounded result. To test its sensitivity, we recompute independence and A_D on eight domains (Table 1), totaling 3,499,776 configuration–item pairs.

Soundness and maximality hold on all eight checked domains. This is evidence for, not a proof of, a generalization beyond them. Coverage ranges from 75.92% to 99.28% and residue from 0.72% to 24.08%, a 33-fold spread. Any synthetic coverage figure is a property of its domain and must not

Table 1. Finite-domain checks. Soundness and maximality hold on each listed domain; coverage varies with the domain.

domain	pairs	snd	max	cov.%	res.%
baseline	331,776	✓	✓	97.70	2.30
$N=2$	4,608	✓	✓	80.40	19.60
$N=4$	262,144	✓	✓	75.92	24.08
bases off bounds	786,432	✓	✓	99.28	0.72
factors $\{0, 1, 3\}$	1,119,744	✓	✓	78.69	21.31
widths $16\times$	331,776	✓	✓	96.36	3.64
coprime widths	331,776	✓	✓	98.77	1.23
min = max	331,776	✓	✓	90.24	9.76

Table 2. Study A: observed upper bound and coverage on the same 2,255 eligible containers from 1,200 pages (six widths, 320–1920 px). Unsourced intermediate vocabularies are not presented as measured gains.

observed oracle: no coupling detected	45.76% (1,032/2,255)
item-local: all items admitted	28.07% (633/2,255)
item-local + mode assumption	not evaluated
item-local + aggregate-space assumption	not evaluated
oracle minus predicate coverage	17.69 percentage points (399)
model reproduces browser within 0.5 px	99.98% (34,078/34,086)
items browser-labelled independent	67.66% (3,844/5,681)
precision (admits that were independent)	100.00% (3,004/3,004)
recall	78.15% (3,004/3,844)
node-weighted decomposable surface	30.01%

be quoted as a constant; our earlier draft incorrectly emphasized baseline coverage of 97.70% as if it characterized the condition generally.

The min = max domain also refuted an earlier form of the condition, which compared the raw flex base size to the bounds rather than the hypothetical size. That form is sound but not maximal: it rejects four always-independent shapes in which the bounds coincide and the basis lies outside them. The baseline domain hid this because its minima and maxima never coincide, so the relevant clause fired zero times and the simplification looked lossless.

3.5 Real pages: observed oracle, then predicate coverage

Start without the predicate. In 1,032 of 2,255 containers (45.76%), no item is observed to move under any recorded sibling probe at any sampled width. We call this the *observed oracle*: if the probes accurately implement the target perturbation contract, any sound independence condition must reject the 1,223 containers with observed coupling (54.24%). A failure to observe movement is not evidence of independence under every possible edit, so even the 45.76% is an optimistic upper bound for that contract. It is conditional on the corpus, missing-asset rendering, eligibility filters, pinned dimensions, probe family, and tolerance. It does not bound contracts that restrict the allowed edits or express dependencies rather than independence.

There are zero observed false admits among 3,004 admissions. A false admit would invalidate applying the predicate in that measured setting, but zero false admits under finite probes is not a soundness proof over all real pages.

Read the container rates. Under our all-items-independent decomposition, one failed item prevents accepting the whole container. Other proof strategies may still benefit from partial facts

Table 3. The 840 misses, using the saved scorer’s raw-basis comparisons at 320 px. L means $b \leq \min$; U means a finite maximum with $b \geq \max$. These are descriptive first-width categories, not the clamped Hyp predicate or causal attributions.

grow	shrink	L	U	items	share (%)
0	> 0	no	no	603	71.79
> 0	> 0	yes	no	133	15.83
> 0	> 0	no	no	46	5.48
0	> 0	yes	no	31	3.69
0	> 0	no	yes	15	1.79
> 0	0	yes	no	8	0.95
> 0	0	no	no	4	0.48

or relational contracts. Of the 71.93% of containers not admitted, 54.24 percentage points have observed coupling and 17.69 points have no observed coupling but are not covered by the predicate. Thus the unadmitted containers are not all structurally unreachable: that interpretation conflates these sets and extrapolates a bounded result to a different domain.

The missing vocabulary ladder. Saved inputs permit recomputing mode, initial free space, and the sum of hypothetical sizes for the unperturbed container. They do not make these facts invariant under sibling changes: all three can change when a sibling’s basis changes. Simply admitting items using those baseline values would repeat the refuted candidate, not measure a sound improvement. A richer predicate must establish the relevant assumption throughout the allowed edits, or state a restricted rely condition and rescore the oracle for that changed contract. We have not derived and validated those intermediate predicates; Table 2 therefore gives measured endpoints, not a coverage curve. Finite-domain maximality alone cannot settle the 399-container gap on real pages.

Which items are missed? The saved v3 score attributes 2,703 of 3,004 admissions (89.98%) to the shrink-floor facet. More instructively, Table 3 accounts for all 840 browser-independent items not admitted. The dominant 603-item class (71.79%) has zero grow, positive shrink, and a basis above its minimum but below any finite maximum at 320 px. Such a shape suggests testing context-dependent shrink constraints, but these marginal counts do not establish why an item failed at another width or why no probe moved it.

The 31 zero-grow, positive-shrink items with L true illustrate why the width distinction matters: satisfying the predicate at 320 px does not imply admission at all six widths. Diagnosing each rejection at its first failing width is a separate analysis, not available from this aggregated breakdown.

4 Study B: Disclosure Contracts

4.1 The gap

At the study snapshot, the repository carried a small TLA+ publication model with five safety invariants, roughly 150 browser assertions in total, and a hand-maintained model-to-implementation table. The assertions were not all disclosure checks. The implementation was not proved to refine the model, and the existing audit did not cover every emitted sink. This is one self-audited artifact, not an external corpus or a baseline comparison with a dedicated secret scanner.

4.2 Design

One policy file supplies the abstract model and the artifact checker. TLC checks properties over declared sources, sinks, and flows; it sees no emitted bytes. The concrete checker reads eligible text files and text extracted from PDFs, then searches the declared PII flows. Sharing a description

avoids duplicate policy definitions, but does not prove that its labels or flow edges match the build. An undeclared dependency, an unexamined encoding, or an unextractable PDF can still escape a successful check.

Audience forms a total order $\text{public} < \text{flagged} < \text{local}$; personally identifying information is an orthogonal flag. Sources carry labels, sinks carry clearances, and the property is that no published sink carries content above its clearance. The prior model’s “no private content in production” is the single instance where the clearance is public and the source is the local overlay.

Encoding is not removal. Reversal, base64, HTML entities, and percent-encoding preserve information. Our removal-style declassification contract therefore requires both a declaration and a check that the known value is no longer detectable. This is a design restriction of this checker, not a definition of all declassification: an explicitly authorized disclosure may retain information. We represent permission to retain contact information by a sink-clearance override with a written reason, separately from a claimed removal.

Three checks implement this design. **C1** walks the output tree subject to file-type and generated-asset exclusions and reports unenumerated files. **C2** searches each enumerated sink’s declared PII sources using the implemented detectors. For the loaded email value it checks five listed representations; the phone fallback uses a regular expression, rather than all three configured encodings. **C3** rechecks declared removals and reports a contradiction if a detector still finds the content. **C3** prevents a detectable reversal from being excused merely by adding a removal declaration. Absence under these detectors does not prove information destruction, semantic confidentiality, or implementation refinement.

4.3 Findings

F1: an email address published in reversed form. The address appears in two published pages inside a data attribute, reversed. The governing assertion is a literal `not.toContain` check, which passes. The information is intact and recovered by string reversal. Notably the same policy is enforced by regular expression at one sink and by literal match at another: one sink is defended against a class, the other against a string.

F2: two published PDFs outside the prior audit set. The historical audit omitted a linked CV PDF and the earlier paper PDF. The CV contains an email and phone number; this is not evidence that both PDFs contain disallowed information. Nor does excluding contact details from one HTML or crawler-facing sink establish a site-wide ban. The finding is an audit-coverage gap requiring an explicit policy decision, not automatically a confidentiality violation.

Neither is a breach; a CV is contact material and the address may be intentional. The defect is that a stated policy is enforced inconsistently across sinks, because tests cover the sinks someone remembered. The remediated policy marks that sink as permitting contact details *with a written reason*. Forcing the decision to be made once and recorded is the engine’s job; making the decision is not.

The model’s remediated instance should not be confused with a remediated build. Applying that policy to the unchanged HTML makes **C3** report the reversed email as contradicting a removal declaration. This is the empirical motivation for checking the declaration against text rather than trusting the policy’s abstract classification.

4.4 Non-vacuity

Recorded execution time is about eight seconds for the five model instances and under two seconds for the artifact check. These figures exclude policy authoring, detector development, build time, and ongoing maintenance. They are not a measurement of total verification cost.

Table 4. Abstract policy instances and mutation witnesses. A pass for the remediated model does not assert that the emitted artifact implements its removals.

instance	result	invariant	witness
current (real repo)	violated	Safety	CV PDF
remediated	holds	—	—
mutant: drop a sink	violated	SinksAudited	CV PDF
mutant: drop declass.	violated	WithinClearance	index page
mutant: drop dev guard	violated	NoLocalInProd	index page

Table 5. Study C paid feasibility result. The baseline rating is not a formal-arm score.

measurement	result
fresh baseline rating	58/100
formal/T rating	unavailable (blocked)
generated / selected contracts	3 / 2
HTML repair candidates	0
provider requests / retries	13 / 0
recorded input / output tokens	188,933 / 161,035
elapsed time	49 min 45 s

5 Study C: Generated Formal Feedback

5.1 Question and protocol

We explored whether task-prompt-generated behavioral models, TLA+ validation, browser replay, and bounded generative repair could improve an LLM-generated HTML application without changing model weights. Contracts were generated from the task rather than selected from a fixed task catalog. This avoids hard-coding expected behavior, but transfers authority to fallible model generation, semantic review, and DOM binding. TLC checks the generated state model; only browser replay can establish that the HTML follows it.

The only completed paid attempt used task 1267, a to-do application requiring category management, date setting, and automatic reminders. It ran a formal arm against a fresh artifact, not a matched formal-versus-nonformal efficacy trial. The local xAI evaluator is also not the published benchmark evaluator. Consequently, neither its rating nor its blocked-arm rate is leaderboard-comparable.

5.2 Live result

The run lasted 49 minutes 45 seconds and recorded 13 provider requests, no retries, 188,933 input tokens, 161,035 output tokens, and 349,968 total tokens. Five specification-preparation calls accounted for 175,378 tokens. Of three generated contracts, category and date contracts survived task adjudication; the reminder contract was excluded. Both surviving models passed bounded TLC checks with six distinct states.

The category binding attempted to activate a category control while a to-do modal intercepted pointer input. The date binding placed the setting and its commit in separate model actions, while the executor required a same-action commit. Error collapsing obscured both causes during the paid run. Formal evidence was therefore incomplete before artifact repair, and no formal score or score delta exists. Later offline changes retained operation-level interception diagnostics, explicit cross-action commit references, pending-setting receipts, and observation-preserving UI context steps.

Exact saved-artifact replay reproduced both failures while leaving the paid artifacts unchanged. These repairs validate harness behavior only; manually annotated bindings are not retroactive benchmark results.

5.3 Incremental-value control

A pre-existing synthetic counter fixture contained a deliberate cap at 10. Shallow executable schedules stopped at depth 8 and passed. TLC generated a 12-action path to the declared coverage target, and browser replay exposed the cap at step 11. We then ran ordinary graph search to depth 12 under the same 12-browser-action trace budget. It produced the identical action sequence and the identical browser mismatch; the repaired fixture passed both traces.

This control demonstrates the value of deeper state-space exploration, not an incremental advantage from TLA+. It uses one intentionally seeded defect and a known target; it is not blind discovery. The paid run likewise provides no positive quality evidence. Study C therefore closes as a negative feasibility result for this pipeline: its specification and binding overhead were substantial, its only paid formal arm was blocked, and its strongest offline defect example was matched by ordinary search. One failed task cannot establish that TLA+ is generally ineffective, but it does not justify further paid runs of this design.

6 Defects in Our Own Instruments

We report these because they are the calibration a reader needs, and because three of the five changed a headline number.

D1: unpinned container (Study A). A content-sized container resizes when a child’s content changes, moving every child through the container’s size rather than through sibling coupling. Since the contract takes the container’s inner main size as given, this mislabels structurally independent items as coupled.

D2: unpinned cross axis (Study A). The same failure one axis over: a wide probe widens a content-sized container, changing an item’s cross size, hence its content-based main size and its automatic minimum.

D3: transition interference (Study A). The mechanism and required measurement controls appear in §2. This affects probes whose relevant properties are transitioning, not every browser-based measurement. No per-page transition ablation was retained.

D1 and D2 can under-report independence through container feedback; transition interference and degenerate containers can inflate apparent independence. The corrected pipeline reports 67.66% instead of the previously circulated 73.26%. These summaries are not a controlled D3-only comparison: the measurement procedure and exclusions changed. We therefore do not attribute the difference to transitions alone or report it as a paired effect size.

D4: spec mutants are vacuous here (Study B). We first mutated the transition relation — publish without auditing, audit ignoring the PII flag, publish with no audit. All three passed. These invariants are properties of the policy–artifact *pair*, not of the transition relation, so against a policy in which nothing is over-clearance, removing a guard exposes nothing. Mutation had to move to the policy.

D5: a tautological invariant, inherited. Our “no local content in production” invariant could not fail, because the definition it quantified over already removed local sources in production mode. It asserted exactly what the definition had deleted. The prior model has the same latent flaw: its preview action branches on mode, so its corresponding invariant can only restate the branch. *Both models were assuming the thing they claimed to prove.* The fix promotes the guard to a policy constant corresponding to one ternary expression in the build, so a mutant can delete it. Only after this fix does the invariant have a counterexample.

Table 6. Specification inventory from the saved study artifacts. Counts expose obligations; authoring time, maintenance time, and comparable total effort were not measured.

disclosure obligation	count
source labels (PII-labelled subset)	5 (2)
sink declarations / source-to-sink edges	12 / 11
prior / remediated audited sinks	10 / 12
detector rules / configured encoding entries	2 / 8
removal declarations / clearance overrides	2 / 1
geometry predicate surface	count
item inputs: basis, grow, shrink, min, max	5
Boolean definitions / nonblank TLA+ lines	3 / 4

D4 and D5 were both caught by keeping a mutant convention that the prior work had established and that it would have been easy to treat as ceremony.

7 Discussion

7.1 Two verification standards, not a league table

The geometric target is a universal independence property within a specified model domain; the browser study tests its application with finite probes. The disclosure result combines finite model checking of a hand-written policy with detector-based consistency checks on one emitted artifact. The second check has a lower guarantee than full noninterference and depends on manually supplied information. Treating both as equally strong meanings of “verifiable” would make the comparison unfair before considering sample size.

The defensible comparison concerns where each approach stops: observed coupling and vocabulary limits for one independence contract, and policy coverage and representation limits for one disclosure checker. Neither 28.07% predicate coverage nor a short checker runtime establishes that disclosure verification is a better investment. Relational layout contracts remain relevant, and two disclosure findings establish existence, not prevalence.

7.2 Authoring burden is not execution time

We can inventory the specification surface but cannot retrospectively recover time spent writing it (Table 6). The study policy contains 210 physical lines including comment fields, or 191 lines after comment fields are removed and the JSON is pretty-printed at two spaces. Line counts depend on representation and include variant scaffolding; they are not effort estimates. The geometry entry counts only the reusable predicate, not the resolution model, probe implementation, proofs, or per-page input recovery. Comparing these line counts as if they represented equivalent work would repeat the compute-time error.

The 12-sink inventory is the historical study policy, not a claim that the evolving build still has exactly those sinks. A maintenance study should introduce a new source read, a new output, and a representation change, then record authoring minutes, review minutes, changes to declarations, and undetected drift for both contract classes. Until such measurements exist, the reported runtimes describe repeat execution only.

7.3 Experiments needed to support a comparative claim

External disclosure corpus. A follow-up could preselect 30–50 public personal-site repositories with linked PDFs, pin revisions and capture emitted artifacts, and report exclusions and framework mix. Audit omission is assessable only when an existing audit set can be recovered: absence of tests is not evidence of a violated policy. Likewise, PDF contact information is a discrepancy only when an explicit policy applies to that PDF, not merely when an HTML sink hides a value. Report these two rates with separate eligible denominators, uncertainty, and false-positive adjudication. Compare pinned Gitleaks or TruffleHog configurations on the same snapshots, distinguishing raw-file scanning from PDF-text extraction and encoding normalization. No such corpus or baseline was run for this paper.

Asset and transition ablations. Re-render the same eligible pages with versioned cached assets preserving intrinsic dimensions, documenting unavailable assets and avoiding arbitrary placeholder sizes. Compare item labels and container rates against the blocked-network condition, including an unchanged-eligibility subset. Independently toggle transition controls on the same snapshots and retain affected-page counts, size-read errors, and label flips. Neither paired experiment is present here; the existing rates cannot be interpreted as unaffected by missing images or as a D3 effect estimate.

Flow inference and replication. Build-graph reachability can flag new imports, but does not automatically recover value-level flows, intended security labels, dynamic reads, or correct declassification. A conservative inference extension needs explicit handling of unknown dependencies and mutation-based recall checks. Human label decisions remain. Independent researchers must perform external replication; another run by the artifact’s owner is not a substitute. Neither automated inference nor independent replication has been completed.

8 Threats to Validity

Study A’s external-request blocking alters image-driven layout, not just its frequency. All reported percentages, including the observed oracle, describe that rendering condition. Eligibility filters and finite, tolerance-based probes can bias or overstate apparent independence; multi-line flex remains outside the target fragment. Eight of 34,086 model/browser observations disagree at 0.5 px tolerance, with a worst discrepancy of 13.505 px; they involve replaced-element aspect-ratio behavior and remain unresolved by the model. Soundness and maximality apply only to the checked finite domains. Bound sensitivity does not convert enumeration into an arbitrary- N theorem.

Study B’s labels and flows are hand-assigned. A mislabelled source is checked against the wrong clearance, and an undeclared dependency is not searched as a flow. Import-graph inference could reduce this omission risk but cannot eliminate it or infer the intended confidentiality policy. The current byte checker does not establish absence of all local-labelled content. Its exclusions, missing-file handling, incomplete detector coverage, and extraction failures further limit the result; in particular, unreadable PDFs are reported but are not themselves included in the current final failure predicate. A fail-closed treatment is needed before such failures can support certification. The historical findings do not require interpreting a checker pass as a complete confidentiality guarantee.

Paraphrase and inference from combinations of public facts are outside the detectors. A semantic detector would need separately labelled benchmarks, false-positive and false-negative measurements, and human adjudication; a model score would not establish sound noninterference. Study B remains one self-audited artifact with no secret-scanner baseline, and neither study measures human effort. Expanding the corpus would improve the evidence but would not by itself prove a systemic cost advantage.

Both studies were designed, run and interpreted by the artifact’s owner. Scripts, counterexamples and the corrections in §5 are offered as inspectable substitutes for independent replication, not as equivalents to it.

9 Related Work

Modular layout proofs verify components against rely/guarantee specifications and compose them for a fragment covering block flow, line layout, floats, and positioned boxes, excluding flexbox and grid [1]. Earlier work formalises CSS layout in SMT and proves accessibility properties across viewport widths [2]. Such contracts may permit container-level assumptions and relational dependencies; they are not limited to our item-local predicate. Study A measures one sufficient independence strategy in an excluded fragment, not an impossibility result for extending those systems.

Lattice-based information flow is due to Denning [5], and language-based enforcement is long established [6]. Recent work composes security-context contracts [7] and applies plan-level flow control to persistent agents [8]. Study B instead checks selected emitted text under an external policy. Its concrete design couples encoding-preserving labels with contradiction checks for declared removals. That distinction is useful against F1, but does not make artifact scanning equivalent to runtime information-flow enforcement or independent of correct source labels. We make no claim of a new calculus or demonstrated superiority to those systems.

10 Conclusion

Executable contracts need an account of both coverage and assumptions. On the measured rendering condition, 45.76% of eligible flex containers have no detected coupling, while 28.07% are admitted by an item-local predicate that is maximal on the checked finite domains. The 399-container gap is not settled by those domain checks. In one repository, encoding-aware inspection and explicit sink auditing expose two coverage defects and motivate rechecking claimed removals. In the generated-artifact feasibility study, the fresh baseline received 58/100 but the formal arm remained unscored after 13 provider requests; ordinary deeper search matched the synthetic TLC witness. Thus Study C demonstrates no quality gain or incremental defect-detection benefit. These findings identify specific limits rather than a cost-effectiveness ranking. The instrument, model, and orchestration defects show why that qualification is part of verification rather than a disclaimer added after it.

References

- [1] P. Panchekha, M. D. Ernst, Z. Tatlock, and S. Kamil, “Modular verification of web page layout,” *Proc. ACM Program. Lang.*, vol. 3, OOPSLA, 2019.
- [2] P. Panchekha, A. T. Geller, M. D. Ernst, Z. Tatlock, and S. Kamil, “Verifying that web pages have accessible layout,” in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2018.
- [3] W3C CSS Working Group, “CSS Flexible Box Layout Module Level 1,” §9.7 Resolving Flexible Lengths. [Online]. Available: <https://www.w3.org/TR/css-flexbox-1/>
- [4] Y. Gui et al., “WebCode2M: A real-world dataset for code generation from webpage designs,” 2024. [Online]. Available: <https://huggingface.co/datasets/xcodemind/webcode2m>
- [5] D. E. Denning, “A lattice model of secure information flow,” *Commun. ACM*, vol. 19, no. 5, pp. 236–243, 1976.
- [6] A. Sabelfeld and A. C. Myers, “Language-based information-flow security,” *IEEE J. Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003.
- [7] C. Zheng and G. Yang, “CONTINUITY: Security-context contracts for composable LLM agent controls,” *arXiv:2609.05269*, 2026.
- [8] D. Girrens and G. Wang, “SPA: Securing persistent LLM agents across queries with plan-first information-flow control,” *arXiv:2608.27234*, 2026.
- [9] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [10] Microsoft, “Playwright.” [Online]. Available: <https://playwright.dev/>